Low-Level Optimization Manual: Bare-Metal Qwen 3.6 Hybrid MoE

Hardware Target: Dual Xeon E5-2680 v4 (Broadwell) + NVIDIA GTX 1080 (Pascal GP104)
Software Paradigm: Zero-abstraction C++/CUDA, Operation Offloading (Horizontal Split)

1. Engine Configuration & Toggleability (The Fallback Contract)

While this engine is hyper-optimized for specific silicon, all hardware-specific exploits must be configurable. This ensures the engine can run in a degraded "compatibility mode" on standard desktop hardware or for unit-testing specific components.

- **Configuration Struct / Macros:** Implement toggles (e.g., `#define ENABLE_NUMA_LOCK`, `config.use_large_pages = true`, `config.enable_rotorquant = true`).
- **Graceful Degradation:**
  - If `ENABLE_NUMA_LOCK` is false, fallback to standard OS thread scheduling.
  - If `MEM_LARGE_PAGES` fails (e.g., missing Windows SeLockMemoryPrivilege), seamlessly fallback to standard 4KB page allocation via `VirtualAlloc`.
  - If `ENABLE_ROTORQUANT` is false, fallback to standard FP16 KV cache (drastically reducing context length but useful for mathematical validation).
  - Provide a toggle to force *all* layers to execute as Attention or SSM for isolated kernel benchmarking.

2. System Topology: The NUMA Fortress

2.1 NUMA Node-0 Isolation

- **The Problem:** Standard inference tools treat the 64GB RAM as a single pool. On dual-socket systems, threads on CPU 1 frequently request data from CPU 0's RAM, triggering the **QPI Penalty**. Moving data across the QuickPath Interconnect introduces ~100ns of latency and caps bandwidth at ~15 GB/s (less than half of local DDR4).
- **The Optimization:** Hard-lock the entire process to CPU 0 using `SetProcessAffinityMask(0x3FFF)` (Windows) or `numactl` (Linux). Cores 0 and 1 are reserved for OS interrupts and PCIe management to ensure the inference workers (Cores 2-13) are never descheduled.
- **Execution:** Use logical HT mappings to ensure physical cores are saturated. On Broadwell-EP, physical cores map to index `i` and `i+14`.

2.2 Large-Page Memory Lockdown (`MEM_LARGE_PAGES`)

- **The Problem:** 16GB of static weights require 4.1 million standard 4KB page translations. This overwhelms the Broadwell **Translation Lookaside Buffer (TLB)**, causing the CPU to stall frequently while searching for physical memory addresses.
- **The Optimization:** Allocate the `WeightPool` and `KVPool` using 2MB Large Pages via `VirtualAllocExNuma`.
- **The Result:** Page translations drop from 4 million to ~8,000. TLB misses are nearly eliminated. Locking the memory physically prevents the Windows kernel from silently paging weights to the other CPU node.

3. CPU Optimization: Broadwell Micro-Architecture

3.1 Hyper-Thread Sibling Asymmetry (Fetcher-Worker Pairs)

- **The Reasoning:** Hyper-threaded logical cores share the same physical execution units (ALU, FMA). If two threads attempt AVX2 math on one core, they stall each other. However, they possess independent Load/Store buffers.
- **The Optimization:** Assign logical Sibling A (Thread 0) to math and logical Sibling B (Thread 14) to prefetching. Sibling B acts as a **Programmable DMA Engine**, executing `_mm_prefetch` instructions to pull data into the L1/L2 cache hierarchy just before Sibling A executes the matrix math.
- **Execution:** Sibling B spins in a `_mm_pause()` loop until a new weight tile pointer is requested.

3.2 L3 Micro-Tiling & SPSC Ring Buffers

- **The Reasoning:** The active MoE expert weights for a single token weigh ~1.12 GB, but the Broadwell L3 cache is only 35 MB. Uncontrolled prefetching causes an **L3 Tsunami**, where weight data flushes out the model's critical activations and KV headers.
- **The Optimization:** Use a Lock-Free Single-Producer Single-Consumer (SPSC) ring buffer to limit the "look-ahead" window. The Fetcher is hard-throttled to keep exactly **16MB** of weight tiles in L3.
- **Execution:** Once the ring buffer has 1 tile, the Fetcher waits until the Worker signals consumption. This ensures the L3 cache is used as a high-speed streaming conveyor belt rather than a static storage pool.

3.3 Non-Temporal Streaming Loads (`VMOVNTDQA`) & Vectorized Unpacking

- **The Reasoning:** Standard loads mark data as "Recently Used" (LRU), polluting the 35MB L3 cache. Furthermore, 3-bit weights do not align with 8-bit bytes, making scalar bit-shifting prohibitively slow.

- **The Optimization:** Load weights using `_mm256_stream_load_si256` to completely bypass the cache hierarchy. Unpack the 3-bit QuIP# weights using the AVX2 Vector Variable Shift intrinsic `_mm256_srlv_epi32`.
- **Execution:** Load a 32-byte chunk, gather 3-byte groups into 32-bit lanes, and use a pre-calculated shift vector `[21, 18, 15, ..., 0]` to instantly align 8 weights simultaneously.

4. GPU Optimization: Pascal GP104 Sequence Math

4.1 Storage-Compute Decoupling (`Q4_to_FP32`)

- **The Reasoning:** The GTX 1080 throttles FP16 compute to 1/64th the speed of FP32. Storing weights in FP32 requires an 8.3 GB fetch (26ms stall).
- **The Optimization:** Store Attention/SSM weights in **4-bit (Q4)** format in VRAM but perform all math in **FP32**.
- **Execution:** The kernel loads 128-bit chunks (`LDG.E.128`), unpacks the 4-bit nibbles to FP32 registers using bitwise masks, and executes the GEMV on the FP32 CUDA cores. This drops the memory payload to ~1.03 GB (3.2ms latency).

4.2 Symmetric RotorQuant (4-bit K / 4-bit V)

- **The Reasoning:** Standard Q4 quantization destroys positional accuracy due to massive "outlier channels".
- **The Optimization:** Apply a **Clifford Cl(3,0) Rotor** (quaternion sandwich product) to the K and V vectors inside the GPU registers before quantization to "smear" the outliers perfectly across all dimensions.
- **Effect:** Because the 3:1 SSM hybrid reduced our KV cache volume by 75%, we use **symmetric 4-bit** for both K and V. This grants maximum match accuracy while maintaining a ~595,000 token context window within the 8GB VRAM envelope.

4.3 Persistent MoE Routing (`__ldcg`)

- **The Optimization:** Load the tiny MoE Router weights via the PTX assembly instruction `ld.global.cg` (Cache at L2).
- **Reasoning:** This permanently locks the router weights in the GP104's **2MB L2 cache**. GDDR5X VRAM is never touched during the routing phase, ensuring the 12-bit expert indices are calculated in microseconds and fired asynchronously to the CPU via pinned host memory.

5. SSM Specific Exploits (The DeltaNet Advantage)

Because Qwen 3.6 utilizes State Space Models for 75% of its layers, it bypasses the traditional memory-bandwidth limits of GPUs.

5.1 L2 Recurrent State Persistence & SRAM Convolution

- **The Optimization:** The fixed 1MB recurrent state is stored in raw **FP32** (no quantization) and flagged for L2 residency. The 4-token temporal history required for the Depthwise 1D Convolution is managed entirely within the **48KB Shared Memory (SRAM)**.
- **The Result:** The GPU updates the state at >1,000 GB/s (L2 bandwidth). The convolution acts as a hardware shift-register inside the SM, avoiding repetitive global VRAM reads. Sequence math becomes essentially "free."

5.2 Taylor Expansion for Softplus

- **The Reasoning:** Pascal GPUs have only 32 Special Function Units (SFU) per SM for transcendental math (logarithms, exponentials).
- **The Optimization:** Replace the SSM Softplus step ($\log(1 + e^x)$) with a rational polynomial Taylor approximation executed on the 128 standard FMA cores per SM.

6. Quality of Life (QoL) Tooling

To deploy this engine seamlessly, two ancillary tools must be created.

6.1 The GGUF Bare-Metal Converter (Python)

Standard GGUF files cannot be parsed by our C++ engine due to our highly specific, split-memory formats.

- **Component:** `tools/convert_gguf_to_baremetal.py`
- **Function:** Uses the Python `gguf` library to read the original Qwen GGUF.
- **Routing:** * Extracts Sequence Backbone tensors (`attn_qkv` , `ssm_conv1d` , etc.) and scales/packs them into our custom **4-bit block-quantized binary blobs** (.gpu_q4 extension).
    - Extracts FFN Experts (`ffn_gate_exps` , `ffn_up_shexp`), applies the QuIP# random orthogonal rotation offline, and packs them into **3-bit bit-shifted binaries** (.cpu_q3 extension).
    - Generates a JSON manifest detailing the exact byte offsets for the C++ `NUMAPool` allocator to ingest.

6.2 Interactive CLI Chat Interface (python)

For debugging, unit testing, and actual use, a robust console interface is required.

- **Component:** `./chat.py`
- **Function:** A wrapper around the `Orchestrator` class.
- **Features:**
  - **Token Streaming:** Implements a callback within `generate_next_token` to instantly `printf` strings to `stdout` as the GPU samples them, rather than waiting for the full sequence.
  - **BPE Tokenization:** A lightweight, dependency-free Byte-Pair Encoding reader that converts user console string input into the initial integer token arrays.
  - **Telemetry Overlay:** A togglable debug mode (`--show-metrics`) that prints the current T/s, KV Cache size, and VRAM/DDR4 utilization at the end of every generation